Online Deployment and Evaluation of Consumption-Aware Memory Allocation for Scientific Workflows

Nagarjuna Chowdary Kandimalla, Bhanu Prakash Vangala, and Tanu Malik
Department of Electrical Engineering and Computer Science
University of Missouri
Columbia, MO 65211, USA
{nkmh5@umsystem.edu, bv3hz@missouri.edu, tanu@missouri.edu}

***Abstract*—Scientific workflows rely on schedulers to reserve memory before tasks start, balancing execution failures against excess reservations. Consumption-Aware Memory Prediction (`CAMP`), introduced in our prior WORKS paper, augments task metadata and memory history with measured and predicted data consumption. This study examines the value and cost of that feedback through component analyses, selective-auditing replay, and new Nextflow/Slurm executions. The component results expose tradeoffs between prediction error, allocation coverage, and auditing cost rather than a uniformly dominant configuration. In a live Bowtie2 evaluation with 6,000 tasks per stage, an additional pass over the original inputs reduces aggregate CAMP requests by 75.34%. On a different ENA study, initial-request coverage falls from 99.58% on repeated inputs to 94.15%, despite continued online learning. Branching directly from the same cold checkpoint, without the extra pass, increases requests by 7.62% and coverage to 95.78%. All tasks complete with no recorded OOMs under a separate 128 MiB per-task execution allowance; these observations do not establish the sufficiency of CAMP requests alone. The results support evaluating consumption feedback through its coverage–reservation tradeoff, while distinguishing online adaptation from frozen-model generalization and allocation estimates from end-to-end resource savings.**

***Index Terms*—scientific workflows, online memory allocation, consumption feedback, eBPF, selective auditing**

I. Introduction

Scientific workflows execute many tasks whose memory requirements vary with their inputs and processing behavior. Workflow engines must request memory before these tasks run. An insufficient reservation can cause an out-of-memory (OOM) failure and a retry; an excessive reservation occupies capacity that could serve other tasks. Predictors based on task metadata and completed executions seek to balance these costs [1]–[3]. Their practical value depends on both allocation safety and the resources reserved to achieve it.

Prelaunch information only partially describes memory demand. A task may read a small fraction of a large staged index, or access file-backed data through memory-mapped page faults rather than explicit read calls. Conversely, a streaming task may process many bytes while retaining little data in memory. Consumption measurements therefore provide contextual evidence rather than a direct measure of peak resident memory. The pending task's consumption is also unavailable at allocation time; only completed observations can inform its request.

Our prior WORKS paper introduced Consumption-Aware Memory Prediction (`CAMP`) [4]. It combines static task features and historical peak memory with consumption measurements from audited tasks. An auxiliary model predicts consumption before launch; quantile models and residual calibration then produce a memory request. Task-attributed eBPF measurements and completed execution outcomes supply feedback for later allocations. The present paper builds on this architecture to study a practical deployment question: whether consumption feedback can reduce wasted memory while maintaining reliable execution as workflows encounter previously unseen inputs.

The earlier CAMP study compares prediction with and without consumption features and evaluates selected baselines, ablations, and selective-auditing policies [4]. Here, new Bowtie2 executions distinguish repeated-input learning from adaptation to a different input study. A cold execution starts from an offline-trained bundle with no Bowtie2 history; a subsequent warm pass repeats the original inputs. A new-input stage then continues learning on a disjoint ENA study. To examine the value of the extra warm pass, a second branch starts the same new inputs directly from the saved cold checkpoint. These branches share their cold state, but neither freezes learning during the new workload.

Three considerations guide this evaluation. First, the value of consumption feedback depends on both measurement and prediction. Explicit reads and file-backed page-fault activity capture different access behavior; historical consumption and predicted consumption supply different model inputs. Separating these effects requires fixed comparison conditions.

Second, auditing incurs execution cost, while its observations can inform only later tasks. Selecting informative tasks is therefore insufficient by itself: the resulting allocation savings must be considered alongside the cost of collecting their measurements. Coverage and reservation cost must also be assessed together, since fewer underallocations may result from larger requests. Development-only tuning and a common

prelaunch information boundary keep the component studies comparable.

Third, repeated execution and adaptation to new inputs test different properties. Repeating a workload supplies additional observations of familiar inputs; applying the resulting allocator to a different study also changes task behavior. Comparing branches from a shared checkpoint can examine the value of the additional pass, while tracking online updates separately from pre-existing knowledge. Initial-request coverage must be reported alongside actual grants, failures, and retries: an execution allowance can prevent an OOM even when the model request is below peak RSS.

Our evaluation addresses three aspects of practical memory allocation:

- *Measurement and features:* eBPF task filtering and Nextflow integration, explicit reads alone versus reads with file-backed page-fault activity, and four feature variants that add historical consumption, predicted consumption, or both to static features and memory history;
- *Allocation benefit and cost:* downstream allocations under selective auditing, estimated memory-time savings relative to auditing overhead, with coverage and reservation cost evaluated together; and
- *Online deployment and adaptation:* new Bowtie2 cold-to-warm executions, adaptation to a disjoint input study, and a shared-checkpoint comparison with and without the extra warm pass.

CAMP's architecture and previously reported results remain prior work. The extension examines when its additional information is useful and whether the allocation benefit justifies the measurement cost.

Section II states the allocation problem and information constraints. Section III summarizes CAMP and its relationship to WORKS. Sections IV and V describe the experimental methods and results, followed by related work (Section VI) and conclusions (Section VII).

II. Problem Formulation

We retain the prelaunch allocation problem introduced in WORKS [4]. A workflow comprises named processes, each of which instantiates tasks for different samples or input partitions. Before task t starts, the allocator issues an initial memory request $\hat{y}_t$. Its static feature vector $\mathbf{s}_t$ contains workflow and process metadata, software and system context, resource settings, and staged input size. Execution subsequently reveals peak resident set size (RSS), y_t, and duration d_t. If the task is audited, it also supplies a consumption measurement c_t. Neither y_t nor c_t is available for that task's initial allocation.

Let $\mathcal{P}(t)$ contain tasks whose outcomes have been recorded before the allocation decision for t. Peak-memory history $\mathbf{h}_{m,t}$ summarizes compatible observations from this set; consumption history $\mathbf{h}_{c,t}$ uses its audited subset. The prelaunch decisions are

$$\hat{y}_t = f(\mathbf{s}_t, \mathbf{h}_{m,t}, \mathbf{h}_{c,t}), \qquad q_t \in \{0, 1\}, \tag{1}$$

where $q_t = 1$ selects auditing. A budget $B \in (0, 1]$ limits the audited fraction over the specified evaluation horizon. Auditing task t cannot improve the request already issued to it; its measurements can inform only later decisions. In live execution, availability follows completion and ingestion times, rather than launch order. Offline evaluation must state the history construction and outcome-revelation protocol used to approximate this constraint.

The allocation objective is to limit underallocation while avoiding unnecessary reservations. For N evaluated tasks, initial-request coverage is

$$\mathrm{Cov} = \frac{1}{N} \sum_{t=1}^{N} \mathbf{1}[\hat{y}_t \geq y_t], \tag{2}$$

where $\mathbf{1}[\cdot]$ is the indicator function. This is the *first-attempt coverage* measure used in WORKS: retry increases do not enter $\hat{y}_t$. A request below a recorded peak is an allocation diagnostic, not proof of an observed OOM failure. Live studies therefore report task completion, failures, and retries separately.

Coverage is considered alongside aggregate requests $\sum_t \hat{y}_t$ and unused memory-time,

$$W = \sum_{t \in \mathcal{D}} \max(\hat{y}_t - y_t, 0)\, d_t, \tag{3}$$

where $\mathcal{D}$ identifies tasks with valid duration measurements. Using memory in GiB and duration in hours gives W in GiB-hours. This peak-based measure quantifies reservation above observed peak demand; it is not an integral of instantaneous unused RSS. Aggregate requests likewise do not represent simultaneous cluster usage. Missing durations and retry accounting must be stated rather than treated as measured zero cost. When estimating net savings, auditing overhead must be charged in compatible units and distinguished from prediction or retraining cost.

For the live study, both new-input branches update their models, history, and calibration from completed outcomes. They differ in whether a further pass over the original inputs precedes the new workload. This tests the value of additional familiar-input feedback during subsequent adaptation; it does not isolate the effect of updating relative to a frozen allocator. Both branches exclude the current task's outcome from its prelaunch decision.

III. CAMP Background and Relationship to WORKS

CAMP's measurement, prediction, and selective-feedback architecture was introduced in WORKS [4]. We summarize the existing design and its implementation to identify the components examined in this study.

A. Measurement and compatible history

The eBPF tracer is attached to a task launcher and tracks descendants through process-fork events. Kernel handlers discard events from untracked execution contexts, preventing unrelated tasks on the same node from contributing to that

task's measurements. For explicit reads, the implementation associates descriptors with observed file opens and accumulates positive return values from `read` and `pread64`. These are returned bytes, rather than requested buffer lengths; descriptor attribution depends on the file operations observed by the tracer.

A successful `mmap` call establishes a virtual mapping but does not show how much mapped data is accessed. CAMP instead instruments the `filemap_fault` entry point for file-backed page-fault activity. The retained tracer adds 4,096 bytes per attributed event. The resulting consumption signal is explicit-read bytes plus this page-fault byte estimate. It does not deduplicate pages or certify that each entry event completed a successful fault; it is therefore not an exact count of unique bytes consumed, physical disk traffic, or resident memory. This distinction motivates the read-only versus read-plus-mmap comparison. A task that streams data and one that retains an index may have similar consumption but different memory peaks.

Completed observations are organized into compatibility scopes. I_0 and I_1 match task-instance/split-group and input identity, respectively; I_2 selects nearby static-feature signatures; I_3 pools process history. These scopes preserve workflow, version, and system context. I_4 broadens to workflow history and I_5 to system-compatible history. Support thresholds select a usable scope, while summaries from multiple scopes remain features. History scopes identify relevant observations; prediction-model selection is a separate mechanism.

B. Consumption prediction and memory allocation

The Access baseline, denoted A+P in the artifact, uses static features and peak-memory history. CAMP adds historical consumption and a prelaunch consumption estimate:

$$\mathbf{x}_t^{\mathrm{AP}} = [\mathbf{s}_t, \mathbf{h}_{m,t}], \tag{4}$$

$$\hat{c}_t = g(\mathbf{s}_t, \mathbf{h}_{m,t}), \tag{5}$$

$$\mathbf{x}_t^{\mathrm{CAMP}} = [\mathbf{s}_t, \mathbf{h}_{m,t}, \mathbf{h}_{c,t}, \hat{c}_t]. \tag{6}$$

The estimator g learns from audited outcomes. Historical consumption $\mathbf{h}_{c,t}$ describes earlier executions, whereas $\hat{c}_t$ estimates the pending task's consumption. The four feature variants retain A+P alone, add either signal separately, or include both. In the offline training pipeline, cross-fitted proxy predictions supply the training feature: each held-out group is predicted by a model fitted without that group. Measured current-task consumption is not substituted for this prediction.

LightGBM ensembles estimate peak-memory quantiles at $\{0.50, 0.90, 0.95, 0.99, 0.995\}$. The process-tail configuration combines a global central estimate with workflow–process upper-tail models and a global fallback; global-tail configurations also exist in the artifact. Feature comparisons must therefore hold the prediction configuration fixed or explicitly identify configuration differences. Changing features and model selection together cannot isolate the contribution of a feature.

C. Calibration and selective feedback

A development-selected policy chooses a base estimate b_t and a residual quantile. Hierarchical calibration uses log residuals $r_\tau = \log(y_\tau / b_\tau)$ and blends a supported residual scope with a broader parent. Residual support determines this scope independently of the history-feature scope. The correction ρ_t produces

$$\hat{y}_t = \lceil b_t \exp(\max(0, \rho_t)) \rceil_{\mathrm{MiB}}. \tag{7}$$

The adjustment is one-sided and rounded to whole mebibytes. Cross-fitted predictions provide development residuals; subsequently available outcomes support online updates. Calibration remains empirical and provides no unconditional coverage guarantee on shifted inputs.

The selective policy allocates an audit budget using prelaunch risk, information, and discovery signals. Peak-memory outcomes can update history after completion; consumption outcomes enter only when measured. The deployment mechanisms are distinct from this policy: the generic Nextflow audit overlay supports process-name filtering and periodic task selection, while the retained live Bowtie2 configuration audits every task. Thus, the live run does not by itself demonstrate selective-auditing gains.

In the artifact, Nextflow configuration overlays add instrumentation to unmodified nf-core workflow sources. A launch wrapper pauses the target, attaches the tracer, waits for readiness, and resumes execution. Bowtie2's allocation integration additionally defines process-specific memory closures that pass task metadata to the prediction service and return requests to Nextflow for Slurm submission. Completion hooks record outcomes, and an asynchronous learner publishes updated model versions for later predictions. The retained launcher requests model updates after each 128 eligible outcomes; history ingestion and model retraining are separate operations. These additions preserve analytical tool commands but require workflow-specific metadata bindings, deployment configuration, and Linux hosts with BCC/eBPF support and tracing privileges. They are not a transparent allocator for every workflow engine.

D. Prior evidence and extension boundary

WORKS already reports the architecture, Access and Sizey comparisons, selected feature and mmap ablations, and selective-auditing results [4]. Earlier Bowtie2 records provide repeated-input deployment evidence; the new executions are packaged separately in the artifact [5]. The present study develops the component analysis, examines downstream allocation benefit relative to audit cost, and evaluates online adaptation to new inputs and the effect of an additional warm pass from a shared cold checkpoint. Reanalysis of existing measurements is distinguished from new experiments. Routing, calibration, and compatibility scopes are described as background; separate ablation sweeps of these mechanisms are outside the present scope.

IV. Experimental Method

A. Populations and comparison conditions

We combine retained component-study records from the CAMP artifact [5] with new live Bowtie2 executions. Replotting the retained measurements does not imply that their workflows were rerun. The feature and mmap studies use 33,516 tasks from GATK, Minimap2, Sarek, Seqinspector, Taxprofiler, and Viralmetagenome. The fixed seed-1996 split assigns 28,490 tasks to development and 5,026 to evaluation, using chronological ordering within each workflow. Four feature views use static information and peak-memory history (A+P), add consumption history ($\mathbf{h}_c$), add predicted consumption ($\hat{c}$), or include both (full CAMP).

The retained LightGBM ensembles and development-selected allocation policies are preserved. Table II compares each feature view under the same global or process-tail configuration; it is a control for feature interpretation, not a new routing-policy sweep. The original endpoint comparison uses global A+P and process-tail CAMP. Published requests and their retained reproduction are reported separately. The mmap study uses paired task identities and peaks; each arm selects its own allocation policy on development data, without test-time request scaling. MdAPE denotes the median absolute percentage error of the median peak prediction, not the error of the final reservation.

B. Selective feedback and audit accounting

The downstream replay uses a separate 30,478-task cohort: 9,141 development and 21,337 evaluation tasks. Five fixed batches reveal recorded outcomes at batch completion. Selective and uniform-random auditing use equal per-workflow counts at budgets of 5, 10, 20, 30, and 50%, with five sampling seeds (1996–2000) and fixed model seeds. The primary evaluation covers batches 2–5: 17,067 tasks after the first update. This is a batched replay, not a reconstruction of live concurrency. Shared initial audited history is excluded from incremental audit budgets.

A separate accounting study holds the published 5,026-task allocations fixed and charges positive paired auditing overhead:

$$\begin{aligned} C_B &= \sum_{t:q_t=1} \hat{y}_t \max(d_t^{\text{audit}} - d_t^{\text{base}}, 0), \\ G_B &= (W_{\text{AP}} - W_{\text{CAMP}}) - C_B. \end{aligned} \tag{8}$$

Memory and time are converted to GiB and hours. All tasks have paired audit timings, but allocation-saving runtimes exist for only 4,577 tasks; the 449 missing runtimes retain zero weight in the published saving. G_B excludes prediction, retraining, and retry costs and does not estimate the allocation improvement caused by each budget. Recorded 95% intervals use paired grouped bootstrap for downstream differences and bootstrap of five seed-level means for audit accounting.

C. Live deployment and new-input branches

The Warm-Transfer study [6] has three stages: cold, warm on the same inputs, and new input. Each contains 6,000 logical tasks across seven Bowtie2 processes and produces 1,974 aligned chunks. The original manifest selects 38 paired yeast ChIP-seq runs from ENA study PRJEB11501; the new manifest selects 38 paired runs from PRJNA486148 [7], [8]. The new runs were selected using input metadata and compressed size, without execution outcomes: 15.20 versus 15.14 GiB. Their run/sample identities, URLs, and FASTQ MD5 values do not overlap the original study. The inputs are not distribution-matched: 34 new runs have 75-base reads and four have 102-base reads, versus 102-base reads in the original set.

Cold execution starts from the fixed seed-1996 offline bundle trained on 28,490 non-Bowtie2 tasks and its leave-one-workflow-out cold-start policy. It begins with no Bowtie2 history, but learns during execution. Warm starts from the validated cold terminal model and matching history and calibration; new input starts from the corresponding warm terminal state. Each stage uses a fresh workflow directory, without cross-stage task-cache reuse. The Direct-Transfer ablation instead starts the same new manifest from the *exact saved Warm-Transfer cold state*, skipping the original-data warm pass. It does not rerun cold execution. Thus, the branches enter new input with 12,000 versus 6,000 prior Bowtie2 observations, respectively.

All live tasks are audited; selective auditing is evaluated only in the separate replay. Successful completed outcomes become eligible for fitting; failed outcomes remain diagnostics. The asynchronous learner first updates after 128 eligible outcomes and then at 128-observation thresholds. The latest 64 observations are held out from fitting for version-matched calibration; a terminal update includes remaining eligible outcomes. Consumption is measured after completion, whereas the task's allocation uses a prelaunch prediction. Model seeds, allocation rules, and worker settings are shared between branches.

Execution uses three 16-vCPU, 64-GiB workers (50 GiB exposed per worker to Slurm). The main experiment uses a 16-vCPU, 64-GiB control node; the direct cold branch uses an 8-vCPU, 32-GiB control node after downsizing. Both learners retain four CPUs. Nextflow 25.10.7 launches Bowtie2 2.4.2 and samtools 1.20. Slurm grants add a fixed 128 MiB to each CAMP request for execution overhead; this allowance is excluded from $\hat{y}_t$, reported coverage, and W. Task retries are limited to three after the initial attempt. Validation checks logical-task coverage, audit outcomes, BAM integrity, and the consistency of the terminal model, history, and calibration.

D. Interpretation and validity

Logical-task metrics pair the first CAMP request with the final successful peak and runtime. Failed-attempt resource costs, auditing, prediction, and retraining costs are excluded from W; actual grants and attempt records are retained separately. A zero OOM count under the execution allowance therefore does not establish that CAMP's nominal requests are sufficient. The shared cold checkpoint avoids variation from an independent cold rerun, but asynchronous completion order can change later model updates. Control-node downsizing also prevents

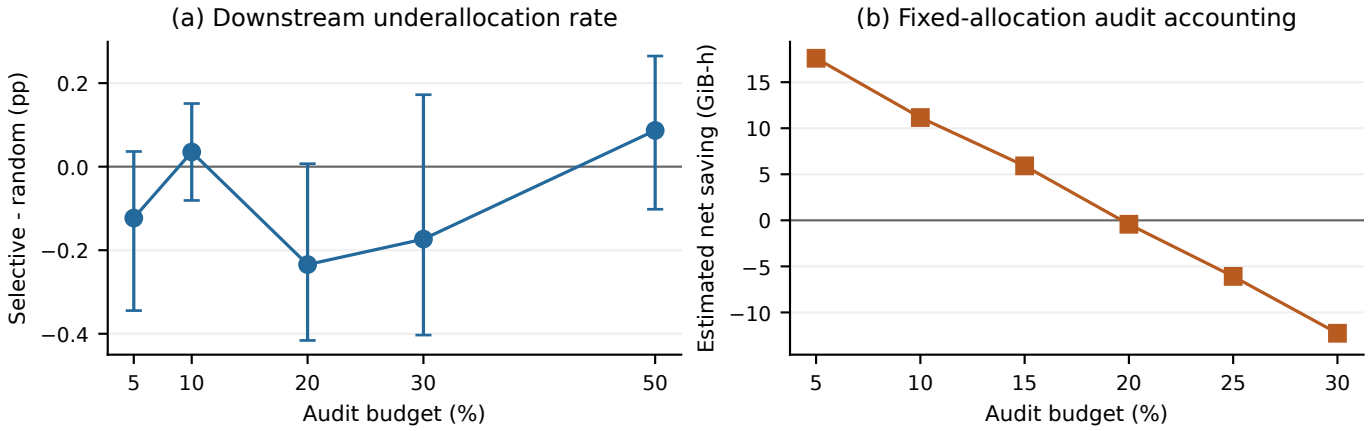


Fig. 1. Two separate auditing studies. (a) Native downstream underallocation-rate difference over 17,067 tasks after the first update; negative values favor selective auditing. Bars are recorded paired 95% bootstrap intervals. (b) Estimated net saving on 5,026 tasks with published allocations fixed (5–30% budgets shown); bars are 95% bootstrap intervals of five seed-level means. Allocation savings in (b) are not caused by the budget choices in (a).

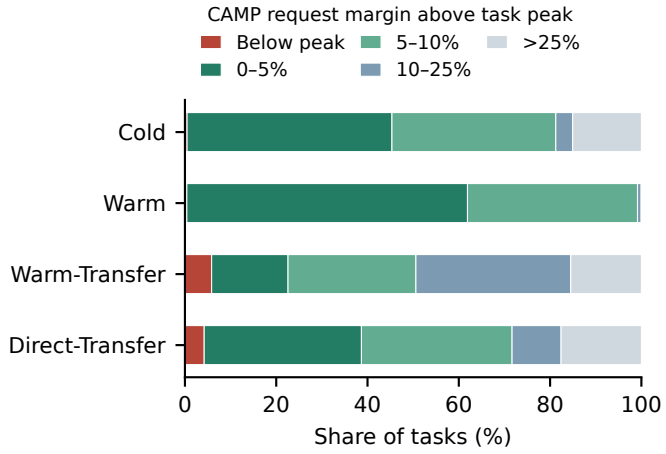


Fig. 2. CAMP request margins in the new live executions: Warm-Transfer cold, warm, and new input after warm; Direct-Transfer new input directly after the shared cold checkpoint. Margins exclude the 128 MiB execution allowance. Negative margins denote nominal requests below task peak, not observed OOMs.

a controlled runtime comparison. The model's system-context features retain the original deployment encoding (13 nodes, 32 vCPUs and 244.14 GiB per node), rather than being refitted to this cluster; conclusions are conditional on this configuration. Only one execution of each branch and one new input study are evaluated, so the results do not establish statistical significance or generalization across studies.

V. Results

We distinguish published WORKS endpoints, reanalysis of retained component measurements, and the new live executions described in Section IV-C.

A. Memory-mapped access and feature contributions

Table I details the read-only versus Read + mmap comparison briefly reported in WORKS [4]. Adding the mmap signal

TABLE I
MMAP ABLATION ON THE SAME 5,026 TASKS. NATIVE DEVELOPMENT-SELECTED REQUESTS; NO REQUEST SCALING.

Signal	Underalloc.	Requests (GiB)	MdAPE (%)
Read-only	32	41,853	22.04
Read + mmap	30	34,271	26.51

reduces underallocations from 32 to 30 and total requests by 18.11%. Median prediction error nevertheless increases from 22.04% to 26.51%. Thus, a better reservation tradeoff need not coincide with lower median prediction error; the two-task difference alone is not evidence of a robust advantage across datasets or random splits.

WORKS reports 87 underallocations for A+P and 31 for CAMP, with total requests of 26,067.45 and 34,277.55 GiB [4]. The retained reproduction gives 85 and 30; these are distinct request realizations, not replacements for the published numbers. Table II expands the four feature combinations using configuration controls. Under global prediction, full CAMP gives 74 underallocations versus 85 for A+P, with 2.06% more requested memory. Under process-tail prediction, A+P attains fewer underallocations (24 versus 30) but requests 603,467 versus 34,271 GiB. The full feature set therefore does not dominate every safety metric. Relative to consumption history alone under process-tail prediction, adding $\hat{c}$ reduces underallocations from 46 to 30 and requests by 57.61%, while MdAPE increases from 21.32% to 26.51%. These comparisons support evaluating feature value jointly with reservation cost.

B. Selective auditing and estimated net saving

At the primary 20% budget, the retained downstream replay gives 265.8 versus 305.8 mean underallocations for selective and random auditing: 13.1% fewer, with 2.4% more requested memory. This preliminary result was also summarized in WORKS [4]. Figure 1(a) shows that the direction varies with budget. At 20%, the paired 95% interval for the rate difference

TABLE II
FOUR FEATURE VARIANTS WITH CONFIGURATION HELD FIXED (5,026 TASKS). U: UNDERALLOCATION COUNT; R: AGGREGATE REQUESTS IN THOUSANDS OF GIB.

	Global		Process-tail	
Feature view	U	R	U	R
A+P	85	26.08	24	603.47
A+P+$\mathbf{h}_c$	81	26.34	46	80.85
A+P+$\hat{c}$	105	26.45	52	123.96
Full CAMP	74	26.61	30	34.27

TABLE III
MAIN LIVE EXPERIMENT (WARM-TRANSFER): 6,000 SUCCESSFUL TASKS PER STAGE. REQUESTS, COVERAGE, AND UNUSED MEMORY EXCLUDE THE EXECUTION ALLOWANCE.

Metric	Cold	Warm	New input
Requests (GiB)	977.79	241.12	319.22
Peak sum (GiB)	229.33	229.38	273.44
Unused memory (GiB-h)	32.86089	0.01847	0.12805
Coverage (%)	99.53	99.58	94.15
MdAPE (%)	1.66	1.13	4.73
Recorded OOMs	0	0	0

is $[-0.416, 0.007]$ percentage points, which includes zero. The five sampling repetitions do not establish uniform improvement or robustness across model seeds.

The fixed-allocation saving is 23.596 GiB-hours before audit cost. Figure 1(b) gives net estimates of 17.597, 11.161, and 5.913 GiB-hours at 5, 10, and 15% budgets. At 20%, the estimate is -0.429 with interval $[-1.077, 0.346]$ GiB-hours. At 25, 30, 50, and 100%, net estimates are -6.087, -12.259, -42.938, and -116.084 GiB-hours. These values identify when the fixed saving covers measured audit cost; they are not end-to-end deployment savings.

C. Repeated-input learning and new-input adaptation

Table III reports the main live experiment. Cold-to-warm execution reduces aggregate CAMP requests by 75.34%, from 977.79 to 241.12 GiB, while peak totals remain approximately 229.3 GiB. Unused memory-time falls from 32.86089 to 0.01847 GiB-hours. Warm requests cover measured peaks for 99.58% of tasks, and median prediction error is 1.13%. These observations indicate more economical requests after repeated-input feedback, conditional on the deployment and accounting boundaries in Section IV-C.

On new input, peak totals rise to 273.44 GiB and requests to 319.22 GiB, increases of 19.21% and 32.39% over warm execution, respectively. Coverage falls to 94.15%, while unused memory-time rises to 0.12805 GiB-hours. The new study therefore exposes undercoverage that is not apparent from the repeated-input result, even though learning continues. Alignment is particularly affected: its median prediction error increases from 0.91% to 15.60%. This process-level result is obscured by the overall new-input median of 4.73%, since indexing and flagstat constitute most tasks and retain different memory behavior. Figure 2 shows the corresponding changes in request margins.

TABLE IV
SHARED-COLD-CHECKPOINT ABLATION ON THE SAME 6,000 NEW-INPUT TASKS: WARM-TRANSFER INCLUDES AN EXTRA ORIGINAL-DATA WARM PASS; DIRECT-TRANSFER PROCEEDS DIRECTLY FROM COLD.

New-input metric	Warm-Transfer	Direct-Transfer
Requests (GiB)	319.22	343.56
Peak sum (GiB)	273.44	277.96
Unused memory (GiB-h)	0.12805	0.22109
Coverage (%)	94.15	95.78
MdAPE (%)	4.73	2.69
Alignment MdAPE (%)	15.60	18.96
Recorded OOMs	0	0

D. Effect of the additional warm pass

Table IV compares the new-input branches from the shared cold checkpoint. Without the extra warm pass, CAMP requests 343.56 GiB rather than 319.22 GiB, an increase of 7.62%. Peak totals differ by 1.65%, whereas unused memory-time increases by 72.65%, from 0.12805 to 0.22109 GiB-hours. Coverage nevertheless improves from 94.15% to 95.78%, and overall median error falls from 4.73% to 2.69%. Alignment median error moves in the other direction, from 15.60% to 18.96%; an overall median alone does not establish improvement for every process.

The extra warm pass thus yields lower requested capacity and unused memory-time in this execution, but lower new-input coverage. It does not dominate the direct branch on every metric. Both new-input stages complete all tasks with no recorded OOMs and no duplicate successful observations. All 1,974 flagstat summaries match exactly between the new-input branches; this checks summary agreement, not byte identity of the BAM files. The separate execution allowance prevents interpreting this as evidence of OOM-free operation using nominal CAMP requests alone. Because the branches continue learning and the control node differs, this is neither a frozen-model generalization test nor an estimate of a universal causal benefit from additional warm training. Prediction latency and full retraining resource costs are not isolated; no end-to-end net-saving claim follows from these logical-task memory-time measures.

VI. RELATED WORK

Online workflow allocation. Witt et al. [1] connect execution measurements, resource prediction, and batch scheduling through feedback. Sizey [3] trains multiple predictors and selects models online using completed-task observations. Ponder [2] adapts its sizing rule to available history and the relationship between input size and peak memory, combining regression with historical bounds and fallback requests. CAMP incorporates consumed-data history and predicted consumption alongside task attributes and historical memory measurements.

Distributional prediction. NGBoost [9] predicts parameters of a conditional outcome distribution, enabling uncertainty

estimates beyond a point prediction. CAMP instead estimates selected memory quantiles and applies empirical residual calibration. Neither a distributional output nor a lower median prediction error alone establishes a better allocation policy: the reservation quantile and calibration determine the coverage–cost tradeoff. Our component studies assess coverage and reservation cost together when examining the contribution of consumption features.

Relationship to CAMP. WORKS introduced consumption-aware prediction and selective auditing [4]. This paper treats that architecture as prior work and examines its operational consequences through retained component comparisons, downstream feedback, audit-cost accounting, and live deployment records. The new-input branches examine the effect of an additional familiar-input pass during online adaptation, rather than comparing frozen and updated models or establishing superiority over external allocators.

VII. Conclusion

Consumption feedback must be assessed through both allocation coverage and reservation cost. The retained component studies show that adding measurement or prediction signals does not improve every metric, while audit accounting limits when a fixed allocation saving covers measurement cost. The new live Bowtie2 executions extend this evidence beyond repeated inputs. A warm pass reduces aggregate requests by 75.34%, but new-input coverage subsequently falls to 94.15%. Skipping that pass from the same cold checkpoint increases requests by 7.62% and coverage to 95.78%, exposing a trade-off rather than a uniformly preferable training path. All tasks complete under a separately recorded execution allowance; nominal request sufficiency and end-to-end resource savings remain unestablished. Repeated trials across input studies, matched control-node resources, and a frozen-model arm are needed to isolate these effects and determine how broadly they hold.

Acknowledgment

This work is supported by NASA under grant NASA-AIST-21-0095.

References

[1] C. Witt, D. Wagner, and U. Leser, "Feedback-based resource allocation for batch scheduling of scientific workflows," in *2019 International Conference on High Performance Computing and Simulation (HPCS)*. Dublin, Ireland: IEEE, 2019, pp. 761–768.

[2] F. Lehmann, J. Bader, N. De Mecquenem, X. Wang, V. Bountris, F. Friederici, U. Leser, and L. Thamsen, "Ponder: Online prediction of task memory requirements for scientific workflows," in *2024 IEEE 20th International Conference on e-Science (e-Science)*. Osaka, Japan: IEEE, 2024, pp. 1–10.

[3] J. Bader, F. Skalski, F. Lehmann, D. Scheinert, J. Will, L. Thamsen, and O. Kao, "Sizey: Memory-efficient execution of scientific workflow tasks," in *2024 IEEE International Conference on Cluster Computing (CLUSTER)*. Kobe, Japan: IEEE, 2024, pp. 370–381.

[4] N. C. Kandimalla, B. P. Vangala, and T. Malik, "CAMP: Consumption-aware memory prediction for scientific workflows," in *Workshop on Workflows in Support of Large-Scale Science (WORKS)*, 2026, accepted for publication; to appear.

[5] N. Kandimalla and T. Malik, "CAMP experimental artifacts and reproduction package," https://anonymous.4open.science/r/IEEE_WORKS_26_ARTIFACT-B98F/README.md, 2026, accessed: 2026-08-08.

[6] "CAMP Big Data Reproducibility Artifact," https://github.com/Nagarjuna-kandimalla/IEEE_BIGDATA_26_CAMP_ARTIFACT, 2026.

[7] European Nucleotide Archive, "Study PRJEB11501," https://www.ebi.ac.uk/ena/browser/view/PRJEB11501.

[8] ——, "Study PRJNA486148," https://www.ebi.ac.uk/ena/browser/view/PRJNA486148.

[9] T. Duan, A. Avati, D. Y. Ding, K. K. Thai, S. Basu, A. Y. Ng, and A. Schuler, "NGBoost: Natural gradient boosting for probabilistic prediction," in *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 2690–2700.